reduces the latency of requests to your API controller while ensuring that the data remains fresh.

NOTE An alternative approach might add caching to your strongly typed client, `ExchangeRatesClient`. The downside is that when you need to update the rates, you will have to perform the request immediately, making the overall response slower. Using a background service keeps your API endpoint consistently fast.

You can implement a background task using the `IHostedService` interface. This consists of two methods:

```
public interface IHostedService
{
    Task StartAsync(CancellationToken cancellationToken);
    Task StopAsync(CancellationToken cancellationToken);
}
```

There are subtleties to implementing the interface correctly. In particular, the `StartAsync()` method, although asynchronous, runs inline as part of your application startup. Background tasks that are expected to run for the lifetime of your application must return a `Task` immediately and schedule background work on a different thread.

WARNING Calling `await` in the `IHostedService.StartAsync()` method blocks your application from starting until the method completes. This can be useful in some cases, when you don't want the application to start handling requests until the `IHostedService` task has completed, but that's often not the desired behavior for background tasks.

To make it easier to create background services using best-practice patterns, ASP.NET Core provides the abstract base class